



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA

Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Pruebas SCA + SAST DAT – Levantamiento de la API

ESPACIO ACADÉMICO:

Seguridad en el Desarrollo de Software

PRESENTADO A:

Sergio Arley Puerto Moreno

AUTOR:

Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofia Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

Pruebas SCA + SAST DAT – Levantamiento de la API

Link del repositorio: <https://github.com/JuanK2550/API-TEATRO>

1. Qué se probó

API_TEATRO es una API REST hecha con Node.js y Express para el Teatro Maldonado de Tunja. Gestiona asistentes, eventos, localidades, funciones y boletas. Los datos están en memoria y los 38 endpoints están documentados con OpenAPI 3.0.3.

El SCA revisa las dependencias, el SAST nuestro código sin ejecutarlo y el DAST la API funcionando. Al final añadimos pruebas manuales, porque ninguna herramienta puede saber cuáles son las reglas de negocio de un teatro

2. SCA – Dependencias

Herramienta: npm audit, incluido en npm 11.12.1.

npm audit

npm audit fix

npm ls --depth=0

Los tres tardan pocos segundos. No hay ninguna vulnerabilidad en los 143 paquetes, así que npm audit fix no cambia nada.

found 0 vulnerabilities

3. SAST – Código

Herramientas: Semgrep 1.172.0 y una revisión manual de src/.

semgrep --config p/javascript --config p/nodejs --config p/owasp-top-ten src/

Semgrep corrió 73 reglas sobre 30 archivos sin hallazgos.

Ran 73 rules on 30 files: 0 findings.

La revisión manual sí encontró dos cosas que las reglas genéricas no detectan. Las dos están corregidas.

SAST-01. Los cuatro campos de texto libre solo validaban longitud, así que aceptaban etiquetas HTML y las devolvían tal cual. Sería XSS almacenado para un cliente web que las renderice. Ahora rechazan los caracteres < y >.

SAST-02. Las peticiones mal formadas se registraban como fallos del servidor, así que cualquiera podía llenar el log de ruido y enterrar un fallo real. Ahora los 4xx se registran como aviso y los 5xx como error.

4. DAST – API en ejecución

Herramienta: OWASP ZAP 2.17.0 en modo sin interfaz, el 16 de septiembre de 2026. No navegó la

API a ciegas: importó la definición openapi.json que ella misma genera, y así conoció los 38 endpoints antes de atacarlos.



Figura 1. Swagger UI de la API. Los 38 endpoints de Asistentes, Eventos, Localidades, Funciones y Boletas que ZAP importó para el escaneo.

El Active Scan tardó 2 minutos y 6 segundos sobre 39 URLs. Lo corrimos con RATE_LIMIT_MAX=100000, porque con el límite normal de 100 peticiones ZAP habría recibido 429 todo el tiempo. Al terminar volvimos a dejarlo en 100.

Alertas:

Dos alertas, un alta y una informativa. Las dos las reprodujimos a mano y las dos son falsos positivos.

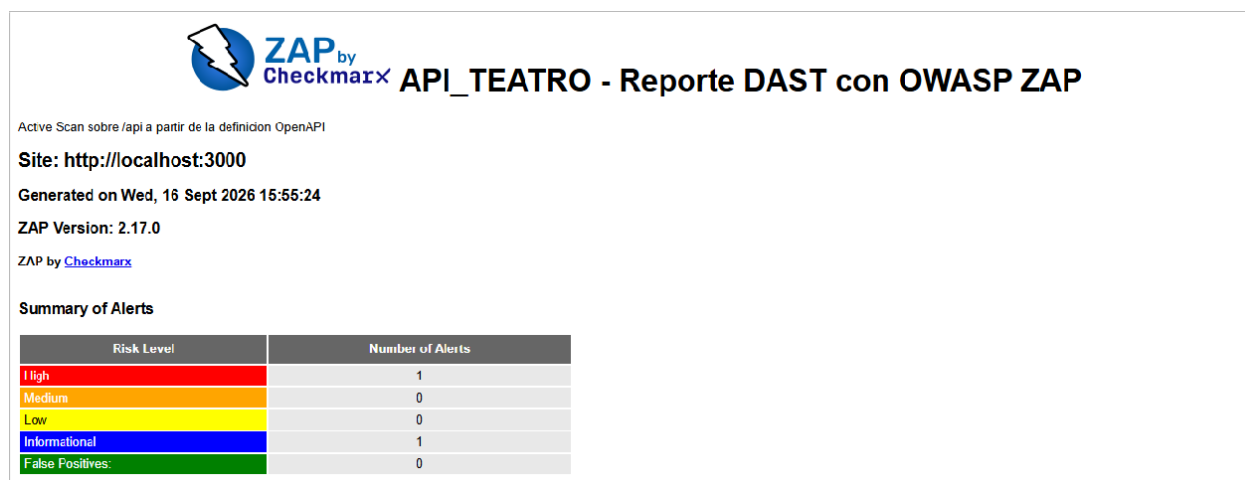


Figura 2. Resumen del informe de ZAP: 1 alerta High, 0 Medium, 0 Low y 1 Informational. La fila "False Positives: 0" es el contador interno de ZAP, que solo cambia si las alertas se marcan dentro del programa.

El 41 % de las respuestas fueron 2xx y el 58 % 4xx. Ningún 5xx: ni un solo ataque provocó un error interno.

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	1
Info	Informational	http://localhost:3000	Percentage of responses with status code 2xx	41 %
Info	Informational	http://localhost:3000	Percentage of responses with status code 4xx	58 %
Info	Informational	http://localhost:3000	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://localhost:3000	Percentage of endpoints with method DELETE	12 %
Info	Informational	http://localhost:3000	Percentage of endpoints with method GET	30 %
Info	Informational	http://localhost:3000	Percentage of endpoints with method PATCH	23 %
Info	Informational	http://localhost:3000	Percentage of endpoints with method POST	12 %
Info	Informational	http://localhost:3000	Percentage of endpoints with method PUT	12 %
Info	Informational	http://localhost:3000	Count of total endpoints	39

Figura 3. Insights del informe de ZAP. El aviso de una advertencia corresponde a la regla DOM XSS, que no se ejecutó porque ZAP no pudo abrir su navegador interno. Esa regla analiza páginas HTML y esta API solo devuelve JSON.

Path Traversal, falso positivo

High	Path Traversal
Description	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal. Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special characters sequences. The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%0%a") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters ("%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character. Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.
URL	http://localhost:3000/api/eventos/10
Node Name	http://localhost:3000/api/eventos/10 ({{titulo,tipo,descripcion,duracionMinutos,clasificacionEdad}})
Method	PUT
Parameter	duracionMinutos
Attack	10
Evidence	
Other Info	
Instances	1
Solution	Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue." For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions. Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as "." and "/" which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a "." inside a filename (e.g. "sensitiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised. Inputs should be decoded and canonicalized to the application's current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked. Use a built-in path canonicalization function (such as <code>realpath()</code> in C) that produces the canonical version of the pathname, which effectively removes "../" sequences and symbolic links. Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations. When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs. Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software. OS-level examples include the Unix <code>chroot</code> jail, <code>AppArmor</code>, and <code>SELinux</code>. In general, managed code may provide some protection. For example, <code>java.io.FilePermission</code> in the <code>Java SecurityManager</code> allows you to specify restrictions on file operations. This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.
Reference	https://owasp.org/www-community/attacks/Path_Traversal https://cwe.mitre.org/data/definitions/22.html
CWE Id	22
WASC Id	33
Plugin Id	6

Figura 4. Detalle de la alerta Path Traversal: PUT /api/eventos/10, parámetro duracionMinutos, ataque "10", evidencia vacía.

La evidencia está vacía: ZAP no encontró el contenido de ningún archivo. La regla compara la respuesta al valor 10 con la de un nombre de archivo cualquiera, y difieren porque 10 es una duración válida y el texto no pasa el validador.

Repetido a mano, los payloads ../../../../etc/passwd y c:\Windows\system.ini devuelven el mismo 400. Además, la API no toca el sistema de archivos: ningún archivo de src/ importa fs, path ni child_process.

User Agent Fuzzer, falso positivo

Informational	User Agent Fuzzer
Description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
URL	http://localhost:3000/api/eventos
Node Name	http://localhost:3000/api/eventos ()({titulo tipo, descripcion, duracionMinutos, clasificacionEdad})
Method	POST
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
Evidence	
Other Info	
URL	http://localhost:3000/api/eventos
Node Name	http://localhost:3000/api/eventos ()({titulo tipo, descripcion, duracionMinutos, clasificacionEdad})
Method	POST
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
Evidence	
Other Info	
URL	http://localhost:3000/api/eventos
Node Name	http://localhost:3000/api/eventos ()({titulo tipo, descripcion, duracionMinutos, clasificacionEdad})
Method	POST
Parameter	Header User-Agent
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
Instances	Systemic
Solution	
Reference	https://owasp.org/wstg
CWE Id	
WASC Id	
Plugin Id	10104

Figura 5. Detalle de la alerta User Agent Fuzzer: tres POST /api/eventos con User-Agent de Internet Explorer 6, 7 y 8.

La regla avisa si la respuesta cambia al cambiar el User-Agent, y en un POST cambia siempre, porque cada llamada crea un evento con otro id. Con un GET, que sí es idempotente, los tres User-Agent devuelven exactamente el mismo cuerpo. La API no lee esa cabecera.

5. Pruebas manuales de lógica de negocio

Ninguna herramienta puede saber que el precio de una boleta lo calcula el servidor, o que una butaca no se vende dos veces en la misma función.

La API ni siquiera pide precio, código ni estado. Eso se implementa con `matchedData`, que descarta cualquier campo no declarado en el validador. Es la protección contra Mass Assignment.

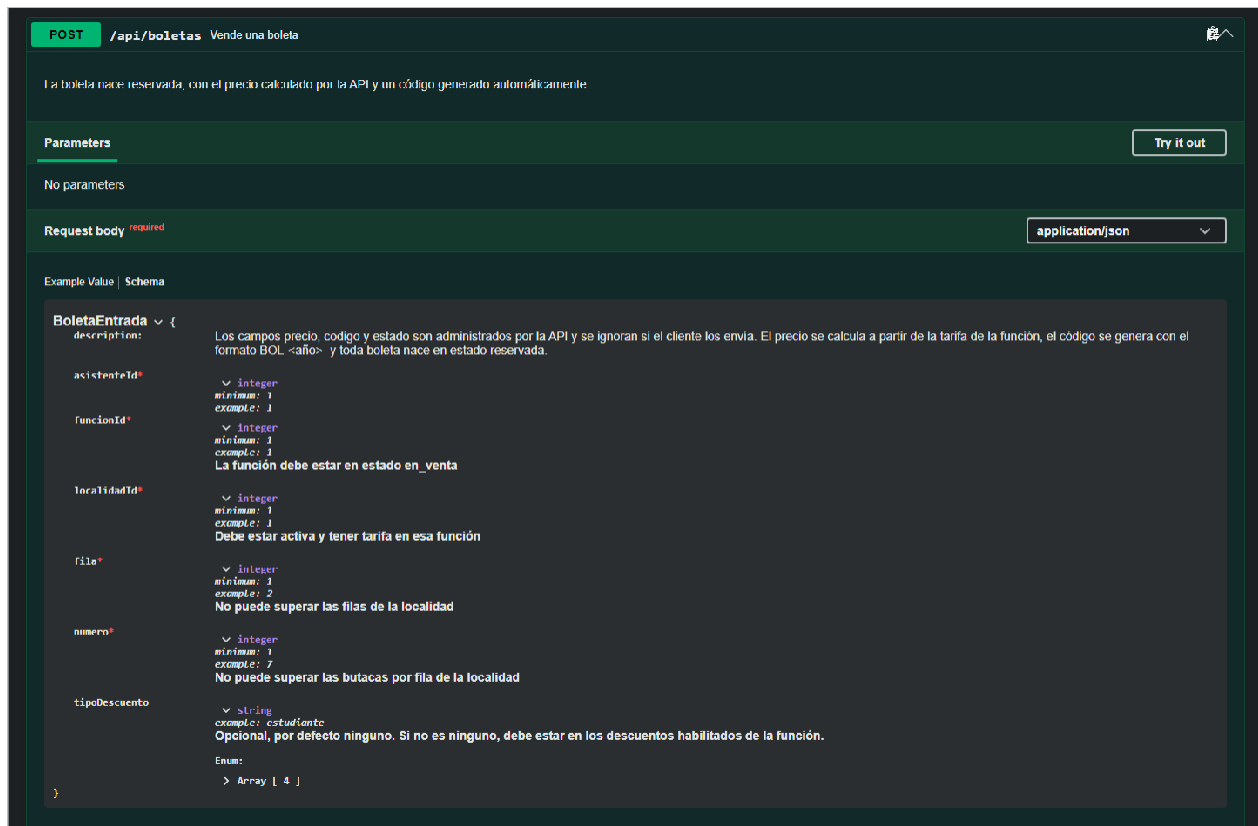


Figura 6. Schema BoletaEntrada en Swagger UI. La entrada solo admite asistenteId, funcionId, localidadId, fila, número y tipoDescuento.

La prueba clave: mandamos una boleta con "precio": 1 y la API respondió 201 guardando el precio real, 75000.

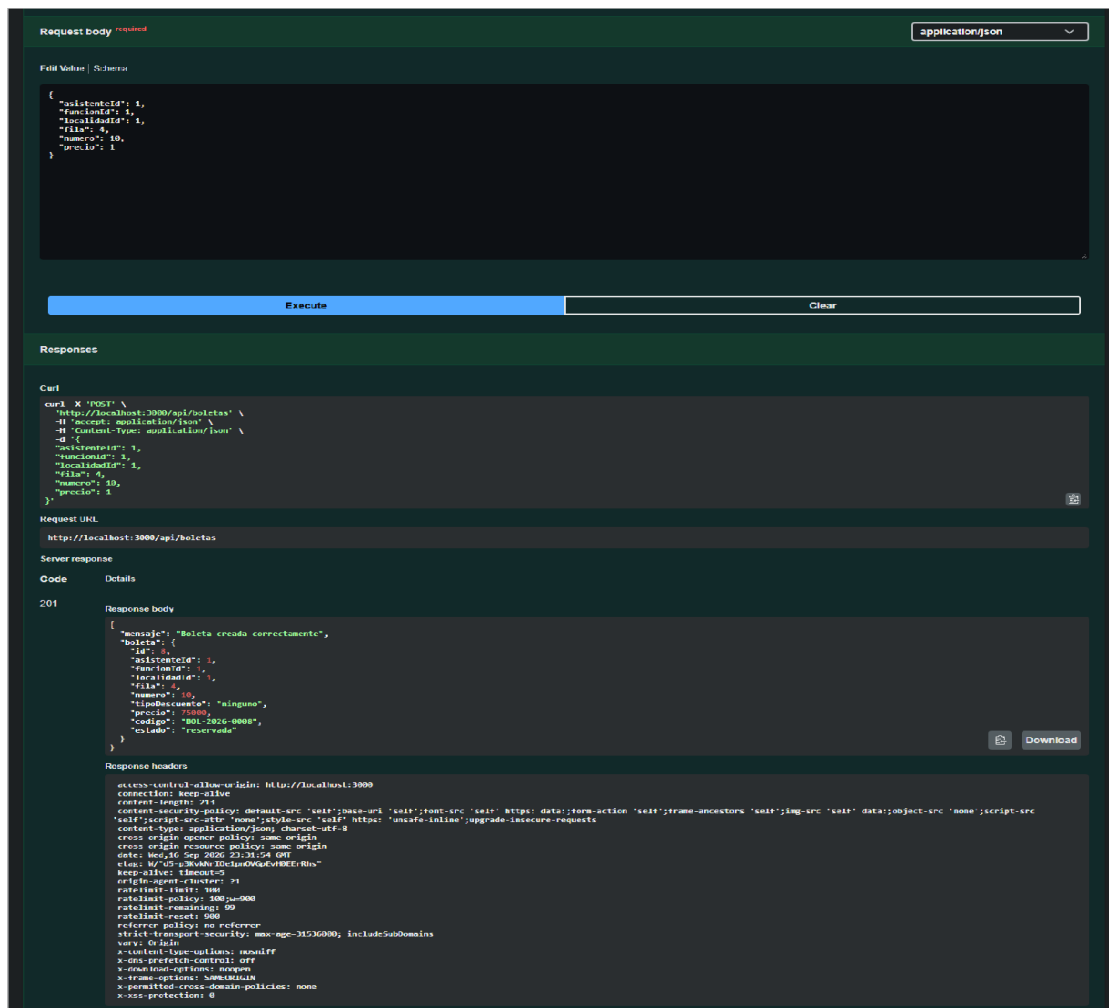


Figura 7. POST /api/boletas con "precio": 1. La API responde 201 con precio 75000, estado reservada y código BOL-2026-0008. En las cabeceras se ven además helmet, el CORS restringido y el límite de peticiones.

Los demás casos

Probados con curl contra el servidor en marcha. Todos dieron el resultado esperado.

- Crear una boleta enviando "estado": "pagada". Nace reservada, el campo enviado se ignora.
- Crear dos boletas para la misma butaca en la misma función. La primera 201, la segunda 409 con el mensaje de que la butaca ya está ocupada.
- Marcar como usada una boleta que ya está usada. Devuelve 409. Y una boleta solo se puede marcar como usada si la función está en curso, que es la validación en la puerta.
- Poner en venta una función que ya está finalizada. Devuelve 409.
- Borrar una función que tiene boletas vendidas. Devuelve 409.
- Crear dos funciones con la misma fecha y hora. La segunda 409, porque el teatro tiene una sola sala.
- Crear una función donde el Balcón cueste más que la Platea Preferencial. Devuelve 409, porque una localidad más cerca del escenario tiene que costar estrictamente más.

6. Resumen

El SCA salió limpio. Semgrep no encontró nada, pero nuestra revisión manual sí, y lo corregimos. Las dos alertas de ZAP eran falsos positivos. Las pruebas manuales pasaron todas.

Lo que podemos afirmar es que en estas pruebas no se encontraron vulnerabilidades. No que la API sea segura sin más: no tiene autenticación, ni HTTPS, ni persistencia, y eso está documentado como limitación en el README